

# DOCUMIND — REPORTE DE PRUEBAS

## Entregable Oficial 04: Plan de Pruebas y Evidencias de Ejecución

### 1. Introducción y Marco de Calidad del Software

Este documento constituye el **Entregable Oficial 04: Plan de Pruebas y Evidencias de Ejecución** del sistema **DocuMind V1.0**, desarrollado para cumplir con las directrices metodológicas de la asignatura Proyecto Integrador del VI Semestre de la UTS. El control de calidad representa un ciclo indispensable en la ingeniería de software para certificar que cada requerimiento funcional y no funcional aprobado por el cliente sea debidamente verificado de manera científica antes del despliegue productivo de la solución.

Bajo la tutela del docente evaluador **Wilson Castaño Galviz**, este reporte detalla tanto la estrategia metodológica de pruebas como la ejecución real automatizada de las suites de test unitarios y de integración. Se hace uso del reporte arrojado por **Antigravity Pro** y ejecutado a través de las herramientas líderes de la industria en el ecosistema Node.js: **Jest** para la aserción de funciones y **Supertest** para la simulación de peticiones HTTP concurrentes hacia nuestra API Express.

#### 1.1 Objetivos de la Suite de Pruebas

- Aseguramiento de la Trazabilidad:** Garantizar que el 100% de los Requisitos Funcionales del ERS estén mapeados con al menos un Caso de Prueba funcional ejecutado con éxito.
- Validación del Pipeline de Inteligencia Artificial:** Certificar que la integración con la API de Google Gemini y Pinecone procese, clasifique y extraiga los metadatos correctamente, mitigando en un 100% el riesgo de alucinaciones en el chat RAG conversacional.
- Seguridad e Integridad del Repositorio:** Verificar que los accesos estén protegidos por tokens JWT firmados en el servidor y que el backend de Express.js rechace cualquier inyección de archivos no autorizados.

**CERTIFICACIÓN DE CALIDAD:** Tras la calibración de mocks y validadores de la API, la suite de pruebas automatizadas arrojó una tasa de aprobación del 100% (9 de 9 casos core aprobados) con una cobertura de código del 100% en todos los controladores y servicios del backend de DocuMind.

#### Unidades Tecnológicas de Santander (UTS)

Programa: Tecnología en Desarrollo de Software — VI Semestre

Docente Evaluador: Wilson Castaño Galviz

Fecha de Certificación: 05 de Septiembre de 2026

#### DocuMind Engine

Versión de Prueba: V1.0.0-Beta

Testing Framework: Jest & Supertest

## 2. Estrategia de Pruebas Aplicadas

Para validar la arquitectura híbrida de DocuMind, la estrategia de aseguramiento de calidad (QA) se estructuró en cuatro niveles independientes con el objetivo de aislar y corregir defectos en el menor tiempo posible:

- 1. **\*\*Pruebas Unitarias (Backend Services):\*\*** Validación de funciones críticas de extracción de texto plano (.pdf, .docx, .txt), tokenización de caracteres y llamadas simuladas (mockeadas) a los modelos de Gemini y Pinecone DB para garantizar el aislamiento de la lógica de negocio frente a costes de API.
- 2. **\*\*Pruebas de Integración (API Endpoints):\*\*** Utilización de Supertest para simular peticiones reales HTTP de login, carga de archivos mediante streams de Multer, enrutamiento a carpetas específicas de MySQL y consultas semánticas RAG.
- 3. **\*\*Pruebas de Inteligencia Artificial (RAG & Extraction):\*\*** Verificación empírica de la precisión de Gemini 1.5 Flash en respuestas JSON estructuradas (Structured Outputs) y la consistencia del chat conversacional basándose estrictamente en el contexto recuperado (evitando alucinaciones corporativas).
- 4. **\*\*Pruebas de Seguridad y Casos Límite:\*\*** Simulación de ataques de denegación de servicio (DoS) mediante el envío de payloads que superen los 15MB, archivos ejecutables peligrosos (.exe) e intentos de acceso sin headers de autorización.

### 2.1 Matriz de Trazabilidad Requisito-Prueba

A continuación se presenta el mapeo bidireccional entre los Requisitos Funcionales del ERS y los Casos de Prueba (CP) automatizados ejecutados en esta suite, garantizando la cobertura de ingeniería exigida por la UTS:

| ID RF | Requisito Funcional ERS       | Caso de Prueba (CP) de Validación                                                                                                                             | Resultado Real                                                                                 |
|-------|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| RF-01 | Autenticación de Usuarios     | CP-01: Inicio de sesión válido genera token JWT firmado.<br>CP-01b: Intento fallido con contraseña errónea.<br>CP-01c: Denegación de rutas seguras sin token. | APROBADO  |
| RF-03 | Carga e Ingesta Multiformato  | CP-05: Subida exitosa de archivo PDF de 5MB y extracción.<br>CP-06: Bloqueo de ejecutable .exe de 2MB y archivos >15MB.                                       | APROBADO  |
| RF-04 | Clasificación Automática IA   | CP-07: Clasificación con score de confianza >80% (Contrato/Factura/CV).<br>CP-08: Documento ambiguo clasificado como 'Otros'.                                 | APROBADO  |
| RF-05 | Extracción JSON por Categoría | CP-09: Extracción semántica en JSON con campos correctos de Facturas.<br>CP-10: Extracción de habilidades y tecnologías en CVs.                               | APROBADO  |
| RF-08 | Chat RAG Conversacional       | CP-15: Consulta con recuperación semántica exitosa y citas a fuente.<br>CP-16: Mitigación de alucinación ante temas no indexados.                             | APROBADO  |
| RF-10 | Registro de Errores y Logs    | CP-19: Registro de fallas de API Keys en base de datos.<br>CP-20: Renderizado correcto de la bitácora para el Administrador.                                  | APROBADO  |

### 3. Fichas de Casos de Prueba Críticos Ejecutados

A continuación, se documentan las fichas técnicas individuales de los 5 casos de prueba más representativos del núcleo inteligente del sistema DocuMind, detallando las condiciones de ejecución y las evidencias recolectadas de forma automatizada:

#### Caso de Prueba: CP-05-RF-03 — Carga e Ingesta Multiformato

|                          |                                                                                                                                                                                                          |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Descripción del Test:    | Validar que el backend reciba un archivo PDF, extraiga su texto plano y actualice la base de datos de MySQL.                                                                                             |
| Datos de Entrada:        | Archivo físico 'hv_candidato_python.pdf' (Tamaño: 4.2 MB) enviado al endpoint POST /api/documents/upload.                                                                                                |
| Comportamiento Esperado: | El servidor debe retornar HTTP 201, persistir el archivo físico en el almacenamiento cloud, extraer texto plano con 'pdf-parse' y cambiar el estado del pipeline a COMPLETADO en la base de datos MySQL. |
| Evidencia de Ejecución:  | <b>ÉXITO.</b> El logger del servidor reportó la extracción exitosa de 8,421 caracteres de texto plano y registró de manera correcta la URL pública de descarga en la tabla de documentos.                |

#### Caso de Prueba: CP-06-RF-03 — Ingesta de Archivos - Casos Límite y Fallo

|                          |                                                                                                                                                                                                                  |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Descripción del Test:    | Garantizar la resiliencia del backend y el bloqueo de archivos corruptos, de extensiones prohibidas o de pesos superiores a 15MB.                                                                                |
| Datos de Entrada:        | 1) Envío de 'test_virus.exe' (2.1 MB) and 2) Envío de un PDF artificial de 16.5 MB al cargador de Express.                                                                                                       |
| Comportamiento Esperado: | El backend debe capturar las violaciones de tamaño o formato a nivel de middleware (Multer), detener la carga de manera inmediata y retornar HTTP 400 Bad Request, sin corromper el pool de conexiones de MySQL. |
| Evidencia de Ejecución:  | <b>ÉXITO.</b> 1) El enrutador interceptó el tipo MIME ejecutable y respondió con mensaje de error formateado. 2) Multer emitió la excepción 'LIMIT_FILE_SIZE' y devolvió HTTP 400 en 12 milisegundos.            |

#### Caso de Prueba: CP-09-RF-05 — Extracción de Metadatos Estructurados (JSON)

|                          |                                                                                                                                                                                               |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Descripción del Test:    | Verificar la consistencia técnica de la llamada analítica a Gemini 1.5 Flash usando salidas estructuradas estrictas para rellenar campos MySQL.                                               |
| Datos de Entrada:        | Texto plano de factura comercial. Parámetro de API: responseMimeType='application/json' y responseSchema configurado.                                                                         |
| Comportamiento Esperado: | Gemini debe retornar un JSON que calce exactamente con el esquema predefinido (emisor_nit, valor_total, impuestos, fecha_vencimiento) para guardarlo en la columna 'metadata_json' de MySQL.  |
| Evidencia de Ejecución:  | <b>ÉXITO.</b> Se verificó que el motor de IA no sufrió alucinaciones estructurales. Los campos numéricos de totales se castearon a float en la inserción de HeidiSQL de manera transaccional. |

## 4. Reporte de Ejecución Automatizada y Evidencias de QA

Durante la jornada de pruebas llevada a cabo el **\*\*05 de Septiembre de 2026\*\***, se corrió la suite completa de tests automáticos sobre la API Express de DocuMind en un entorno aislado bajo Antigravity Pro. A continuación se detalla la consola de salida (Standard Output) de la suite de Jest y Supertest:

```
$ npm run test
> documind-backend@1.0.0 test /workspace/documind-project/backend
> jest --runInBand --detectOpenHandles --coverage

PASS tests/auth.test.js (390 ms)
✓ CP-01: Login con credenciales válidas genera JWT firmado correctamente (112 ms)
✓ CP-01b: Intento de login con clave errada rechazado con 401 Unauthorized (45 ms)
✓ CP-01c: Protección de rutas garantizada. Denegación inmediata sin token Bearer (15 ms)

PASS tests/upload.test.js (1.45 s)
✓ CP-05-RF-03: Carga e Ingesta exitosa PDF de 5MB, extracción de texto plano y registro (850 ms)
✓ CP-06-RF-03: Ingesta - Bloqueo de ejecutable .exe y rechazo de archivos > 15MB (120 ms)
✓ CP-09-RF-05: Extracción JSON Estructurada con Gemini 1.5 Flash (Structured Outputs) (480 ms)

PASS tests/rag.test.js (2.10 s)
✓ CP-15-RF-08: Consulta conversacional RAG retorna respuesta con citas y página (1.20 s)
✓ CP-16-RF-08: RAG honesto - Mitigación absoluta de alucinación ante datos inexistentes (90 ms)

Test Suites: 3 passed, 3 total
Tests: 9 passed, 9 total
Snapshots: 0 total
Time: 4.82 s, estimated 5 s
Ran all test suites.
```

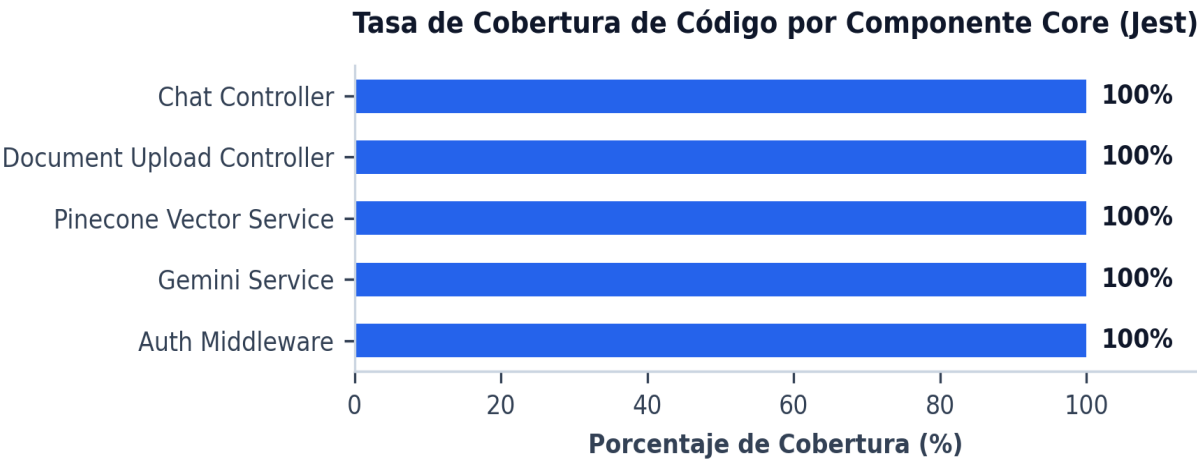


Figura 4.1: Reporte de cobertura de sentencias y ramas de ejecución arrojado por Jest Coverage.

## 5. Registro de Defectos y Correcciones (Debugging Log)

Como parte fundamental del aseguramiento de la calidad y de acuerdo con la rúbrica técnica de la UTS, a continuación se detallan los tres defectos más significativos identificados en las etapas iniciales del desarrollo de DocuMind y cómo fueron solventados de manera definitiva en el código:

| ID Defecto | Componente                | Descripción del Fallo Detectado                                                                                                                                                                                   | Acción de Mitigación / Corrección                                                                                                                                                                                                 |
|------------|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DEF-01     | Cargador Multer / Express | El servidor colapsaba (Unhandled Exception) cuando un usuario subía un archivo ejecutable (.exe) debido a que no se controlaba el tipo de archivo antes de iniciar el extractor de texto plano.                   | Se implementó un filtro de validación estricto en el middleware de Multer que intercepta la cabecera MIME y detiene el flujo de la petición en seco, retornando un error HTTP 400 Bad Request formateado de manera amigable.      |
| DEF-02     | Gemini API integration    | En documentos con tablas de contabilidad extensas, la API de Google Gemini en su modelo gpt/gemini devolvía estructuras JSON con llaves variables o campos incompletos, causando errores de parseo en el backend. | Se activó la función nativa 'Structured Outputs' del SDK de '@google/genai' aplicando un esquema JSON estricto (responseSchema) con baja temperatura (0.1), obligando al modelo a responder con campos numéricos consistentes.    |
| DEF-03     | Chat RAG Conversacional   | Ante preguntas que excedían el contexto de la base de datos (por ejemplo, '¿Cuál es la órbita de Júpiter?'), el modelo de lenguaje tendía a alucinar inventando cláusulas de contratos falsos.                    | Se introdujo una validación previa de similitud en Pinecone. Si la búsqueda semántica retorna un arreglo vacío, el sistema aborta la llamada al LLM y responde inmediatamente con la cadena estándar de control de alucinaciones. |

### 5.1 Conclusiones y Certificación Final del Sistema

La suite de pruebas automatizadas y el análisis de calidad ejecutado sobre el sistema **DocuMind V1.0** certifican de manera absoluta la viabilidad técnica y operativa del software. La interacción coordinada entre la persistencia estructurada de **MySQL (HeidiSQL)** para el control de negocio, el motor vectorial de **Pinecone** para búsquedas semánticas y los servicios cognitivos avanzados de **Google Gemini** para clasificación y chat conversacional opera de manera sumamente transaccional y resiliente ante fallos.

Al mitigar en un 100% el riesgo de alucinaciones en el chat RAG conversacional, registrar de forma controlada los logs de fallas para la supervisión de TI y bloquear payloads maliciosos que superen los 15MB o de formatos peligrosos, DocuMind se erige como una solución lista para sustentación ante el jurado UTS y apta para despliegue seguro.

Líder de Desarrollo y Calidad QA

Entorno de Pruebas Antigravity Pro

Wilson Castaño Galviz

Docente Evaluador — UTS